EXPLAINER · THE LOOP, THE TOOLS, THE SANDBOX, THE HOOKS

How a coding agent actually works

A complete coding agent in plain Node: one HTTP request, eight tools, a loop with four bounds, nine lifecycle hooks, a memory file and a single folder it is allowed to write to. Read it top to bottom, or jump to the part you doubt.

`src/agent.js 215` · `src/prompt.js 81` · `src/tools.js 432` · `src/cli.js 468` · `src/workspace.js 117` · `src/sandbox.js 357` · `src/llm.js 66` · `src/ui.js 164`

CONTENTS

What is in this explainer

01 What an agent actually is 3

02 The shape of the program 4

03 The loop 7

04 The model call 10

05 Messages are the memory 12

06 The eight tools 14

07 edit_file, the interesting one 16

08 The sandbox 17

09 Human in the loop 20

10 The console palette 22

11 Lifecycle hooks 23

12 Long-term memory 28

13 Configuration and precedence 32

14 Bounded autonomy 34

15 A real run, annotated 36

16 Run it yourself 42

17 What it deliberately does not do 44

Page numbers refer to the folio at the foot of each page.

01 What an agent actually is

Strip the branding off a coding agent and three things remain: a **model** that can only produce text, a set of **tools** that can actually do something, and a **loop** that hands the model's own text back to it as an observation. Everything else — retries, sub-agents, skills, permissions, project rules — is a layer laid on top of those three. Memory is the one layer that earns its place early enough to be in this demo: a file the loop reads before step 1 (chapter 12).

mini-coding-js is those three things and two extensions: 2938 lines in `src/`, one dependency (`chalk`, for colour), no SDK, no framework. The model is reached with a single `fetch`. This page walks the whole program, in the order you would read it.

- **2938** — lines in src/
- **215** — of them the loop
- **8** — tools
- **9** — lifecycle hooks
- **1** — memory file
- **104** — offline tests

The whole idea, in six lines

PSEUDOCODE · *text*

```
messages = [system prompt, user task]

repeat up to maxSteps:
    reply = model(messages, tools)          # text in, text out
    if reply has no tool calls:  return reply.content      # done
    for each tool call:  messages.append(tool result)      # observation
```

That is not a simplification of an agent — it *is* the agent. A chatbot is the same program with the loop deleted: one model call, one answer, no way to look at the world and react to what it saw. The rest of this page is the detail behind each line: what the model request contains, what a tool call really is, and what stops the loop from running forever.

What it looks like while it runs

CONSOLE · *stdout*

```
── step 1/12 ──────────────────────────────────────
think: Creating a script, then running it to check it works.
tool → create_file {"path":"demo/hello.js","content":"..."}
result ← created demo/hello.js (72 bytes)
```

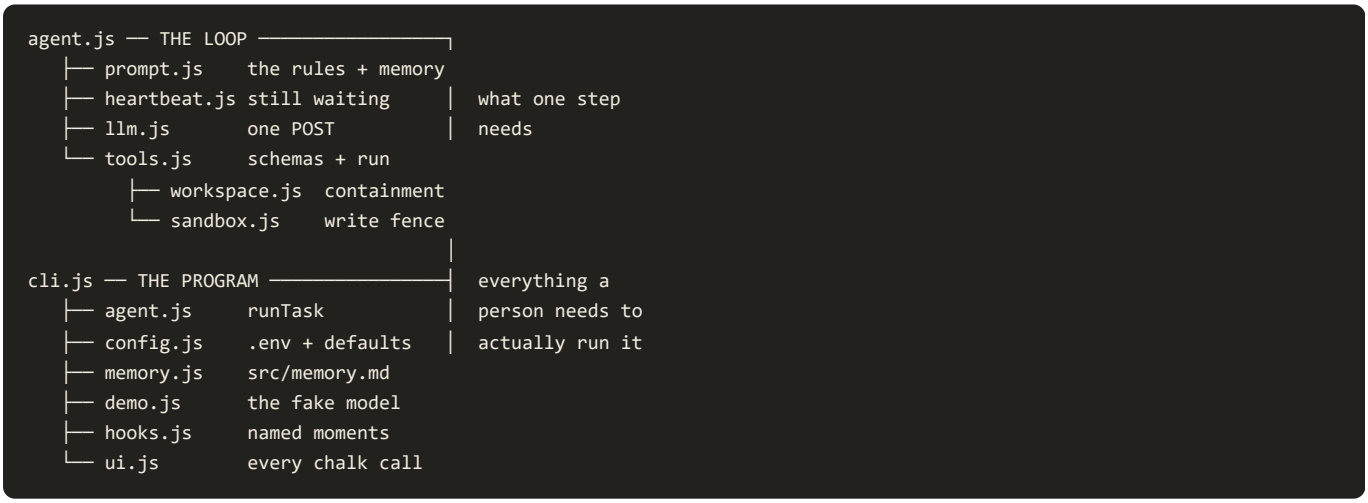
Read those four lines as a sentence: the model *thought*, it *acted*, the world *answered*, and the loop went round again with that answer in hand. Every coding agent you have used prints a variation of this. Nothing in the middle is hidden.

02 The shape of the program

Thirteen files, each answering one question an audience usually asks. They import in one direction only, so there is no cycle to hold in your head — and the loop is a library, not a program: nothing in `agent.js` knows a flag name, a colour or an exit code.

FILE	LINES	THE QUESTION IT ANSWERS
`src/agent.js`	215	**Where is the loop?** `runTask`: seven beats, nine hook call sites, one `for`. Nothing else lives here on purpose.
`src/prompt.js`	81	What is the model told before it starts — and how does memory get in there?
`src/heartbeat.js`	36	Why doesn't a 60-second request look like a crash?
`src/tools.js`	432	What can the model do, and what happens when it asks?
`src/hooks.js`	398	Where can outside code get a turn, and what may it change?
`src/memory.js`	253	What does it know that survives the process?
`src/demo.js`	156	What runs when there is no network, no key and no model?
`src/cli.js`	468	**What does a person type?** Flags, banner, the session loop, y/N prompt, exit codes.
`src/config.js`	195	Which key, which model, which limits — and where each value came from.
`src/workspace.js`	117	How is a path stopped from leaving the sandbox?
`src/sandbox.js`	357	**May the agent change this file at all?** The write fence around save, edit, delete and shell writes.
`src/llm.js`	66	What exactly goes over the wire?
`src/ui.js`	164	Where is the colour? (Nowhere else.)

IMPORTS, TOP TO BOTTOM · *text*



There is no plugin registry, no abstract base class, no event bus. `TOOL_SCHEMAS` is an array literal and `HANDLERS` is an object literal whose keys happen to be the tool names; the dispatcher is thirty lines. To add a ninth tool you would edit two places and be done.

The bottom half of that diagram is the important bit for teaching: `cli.js` imports `agent.js`, never the other way round. The loop can be read, tested and driven in a REPL without the CLI existing — which is exactly what the offline test suite does, handing `runTask` a fake `chatFn`.

The program, in one function

`runCli` is everything a person touches, and it is a straight line: read the flags, resolve the config, build a workspace, a memory store and a `chatFn`, then hand those to `runTask` and turn the result into an exit code. Nothing here touches `messages`, dispatches a tool, or knows what a hook is beyond attaching one — which is why `agent.js` can be read without it.

SRC/CLI.JS (CONDENSED, LOGGING TRIMMED) · *javascript*

```
async function runCli() {
  const opts = parseArgs(process.argv.slice(2));          // flags -> one object
  if (opts.help) return usage();
  if (opts.showHooks) return showHooks();                 // describes the loop, needs no key

  const config = loadConfig({ model: opts.model });        // .env, defaults, precedence
  const workspace = new Workspace(config.workspaceRoot);   // the sandbox the tools live in
  const memory = new Memory({ root: config.projectRoot, file: config.memoryFile, ... });
  if (opts.showMemory) return showMemory(memory, config);

  banner({ config, workspace, memory, commandPolicy, opts });   // one reproducible header

  // The only choice this file makes about the model: real or scripted.
  const chatFn = opts.demo
    ? demoChat()
    : (args) => chat({ apiKey: config.apiKey, model: config.model, timeoutMs: config.requestTimeoutMs, ...args });

  const hooks = opts.noHooks ? null : traceHooks({ log: console.log });

  // Everything above was configuration. This is the agent.
  const result = await runTask({
    task, workspace, commandPolicy,
    maxSteps: opts.maxSteps ?? (opts.demo ? DEMO_SCRIPT.length + 1 : config.maxSteps),
    chatFn, confirm: makeConfirm(), hooks,
    memory: opts.noMemory ? null : memory,
  });

  if (!result.ok && result.reason === 'request_failed') process.exitCode = 1;
  if (!result.ok && result.reason === 'max_steps') process.exitCode = 3;
  if (opts.demo) reportSandbox(result.messages);           // check the guards actually held
}
```

The dependency arrow only points one way. `cli.js` imports `runTask`; the loop has never heard of a flag, a banner or an exit code. Run `node src/agent.js` by itself and it says so: it is a library, and the program is `src/cli.js`.

The fake model

That single ternary above - `demoChat()` instead of the HTTP client - is the whole of `--demo`. It is a `chatFn` with the same return shape as OpenRouter, replaying a fixed script. Every other seam is the real one: the real loop, the real dispatcher, the real sandbox, the real memory tools. The only thing missing is one POST.

SRC/DEMO.JS · *javascript*

```
export function demoChat() {
  let i = 0;
  return async () => {
    const message = DEMO_SCRIPT[Math.min(i, DEMO_SCRIPT.length - 1)];
    i += 1;
    return {
      choices: [{ finish_reason: message.tool_calls ? 'tool_calls' : 'stop', message }],
    };
  };
}
```

Because the reply is data, the script can do something a live model will not do on cue: steps 6–10 deliberately try to escape the sandbox, and `sandboxScorecard()` then reads those results back and prints which guards actually held. That is chapter 15's scorecard — a claim the program checks, not a slide.

> **Read it in this order.** `agent.js` first and in full — it is 215 lines including comments. Then `prompt.js` (what the model is told) and `tools.js` top to bottom, because the schema descriptions are the only prompt engineering in the project. `hooks.js` and `memory.js` are the two chapters you can read last, and the two an audience always asks about. `cli.js` is the program if you need it; `demo.js` is what runs when the wifi does not.

03 The loop

`runTask` is the whole agent. It takes a task, builds the first two messages, and then runs the same seven beats until the model stops asking for tools or the step budget runs out. Here it is with the logging lines trimmed (`agent.js:62–194`):

SRC/AGENT.JS · *javascript*

```
export async function runTask({
  task, workspace, commandPolicy, maxSteps, chatFn, confirm,
  hooks = null, memory = null, log = console.log, heartbeatMs = 5000,
}) {
  // One array is the whole state. prompt.js owns the system message, memory included.
  const messages = [
    { role: 'system', content: buildSystemPrompt({ memory, log }) },
    { role: 'user',   content: task },
  ];
  const ctx = { workspace, commandPolicy, memory, confirm };

  await hook.run('SessionStart', { task, workspace: workspace.root });        // 1
  const submitted = await hook.run('UserPromptSubmit', { task });            // 2

  for (let step = 1; step <= maxSteps; step += 1) {
    log(ui.step(step, maxSteps));
    await hook.run('PreModel', { step, maxSteps, messages });                  // 3

    let reply;
    const beat = heartbeat(log, heartbeatMs);
    try {
      reply = await chatFn({ messages, tools: TOOL_SCHEMAS });
    } catch (err) {
      beat.stop();
      await hook.run('SessionEnd', { ok: false, reason: 'request_failed' });   // 9
      return { ok: false, reason: 'request_failed', messages };
    }
    beat.stop();

    const choice  = reply?.choices?.[0] ?? {};
    const message = choice.message ?? {};
    await hook.run('PostModel', { step, message, finishReason: choice.finish_reason }); // 4

    // A truncated reply means half a tool call. Do not dispatch it.
    if (choice.finish_reason === 'length' && !message.tool_calls?.length) {
      messages.push({ role: 'assistant', content: message.content ?? '' });
      messages.push({ role: 'user', content: 'You were cut off by the length limit. Continue smaller.' });
      continue;
    }

    messages.push(sanitizeAssistant(message));
    const calls = Array.isArray(message.tool_calls) ? message.tool_calls : [];

    if (calls.length === 0) {
      await hook.run('Stop', { answer: message.content ?? '', steps: step });  // 7
      await hook.run('SessionEnd', { ok: true, reason: 'answer' });             // 9
      return { ok: true, answer: message.content ?? '', messages, steps: step };
    }

    for (const call of calls) {
      const name    = call.function?.name ?? '(unnamed)';
      const rawArgs = call.function?.arguments ?? '{}';
      log(ui.tool(name, rawArgs));

      const pre = await hook.run('PreToolUse', { name, args: rawArgs, step });   // 5
      const result = pre.blocked
        ? `DENIED: a PreToolUse hook blocked this call (${pre.reason}).`
        : await runTool(name, rawArgs, ctx);
      log(ui.result(result));

      const post = await hook.run('PostToolUse', { name, result });             // 6
      messages.push({ role: 'tool', tool_call_id: call.id, content: String(post.replace ?? result) });
    }
```

```
  }

  await hook.run('StepLimit', { steps: maxSteps, messages });              // 8
  await hook.run('SessionEnd', { ok: false, reason: 'max_steps' });         // 9
  return { ok: false, reason: 'max_steps', messages };
}
```

Every numbered comment marks a `hook.run(...)` call. Nine names, in the order one run reaches them: the loop is the same seven beats, and each beat now announces itself. Chapter 11 is what those nine lines mean.

The seven beats

#	WHAT HAPPENS	WHERE	HOOK
1	A step heading is printed and a 5-second heartbeat is armed, so a slow provider never looks like a frozen program.	`agent.js:112–120`	`PreModel`
2	One model call: the *entire* message array plus the six tool schemas. Nothing else is sent.	`agent.js:122`	—
3	A truncated reply (`finish_reason: "length"` with no tool calls) is never dispatched as half a call; the model is told to continue smaller.	`agent.js:138`	`PostModel`
4	The assistant message is pushed back into history, stripped to the fields an agent needs.	`agent.js:145`	—
5	No tool calls means the model is finished: print the answer, return `ok: true`.	`agent.js:151–159`	`Stop`, `SessionEnd`
6	Each tool call is dispatched, its result logged, and the string appended as a `role: "tool"` message.	`agent.js:162–187`	`PreToolUse`, `PostToolUse`
7	Loop. The next request now holds the model's own intention *and* the world's answer to it.	`agent.js:112`	—

> **The loop is the agent.** Delete the `for` and you have a chatbot with tools it will never call. Keep it, and the model can look, act, observe and try again — which is the entire difference between "answers questions about your code" and "changes your code".

One more consequence worth saying out loud: `runTask` is one *task*, so `step` counts that call and nothing else. Run a second task in the same session and the counter starts at 1 again with the whole budget available — which is what makes an interactive session (chapter 16) cheap to build: call the same function, hand it the conversation so far.

04 The model call

There is no SDK in this project, on purpose. A tool-calling request to an OpenAI-compatible endpoint is a POST with three interesting fields: `model`, `messages` and `tools`. The whole client is 66 lines (`src/llm.js`).

SRC/LLM.JS · *javascript*

```
export async function chat({ apiKey, model, messages, tools, signal, timeoutMs = 90_000 }) {
  if (!apiKey) throw new Error('OPENROUTER_API_KEY is not set');

  const timeout = AbortSignal.timeout(timeoutMs);
  const abort = signal ? AbortSignal.any([signal, timeout]) : timeout;

  const res = await fetch('https://openrouter.ai/api/v1/chat/completions', {
    method: 'POST',
    signal: abort,
    headers: {
      Authorization: `Bearer ${apiKey}`,
      'Content-Type': 'application/json',
      'X-Title': 'min-coding-js',
    },
    body: JSON.stringify({ model, messages, tools, max_tokens: 2048 }),
  });

  const text = await res.text();
  if (!res.ok) {
    const hint = res.status === 401 ? ' (bad or revoked key)'
      : res.status === 402 ? ' (out of credits)' : '';
    throw new Error(`OpenRouter ${res.status}${hint}: ${text.slice(0, 400)}`);
  }
  return JSON.parse(text);
}
```

Two details are worth a minute on stage.

- `AbortSignal.timeout` **is the entire liveness story at the transport layer.** A socket that never answers becomes an error after 90 seconds instead of a hung agent. The heartbeat in `heartbeat.js` covers the same worry at the human layer: `… still waiting for the model (25s)`.
- **Failures here *throw*; failures in a tool do not.** A bad key, a 402, a dropped connection is not something the model can fix, so it stops the run with exit code 1. A failed edit *is* something the model can fix, so it comes back as text.

Sanitising the assistant message

Providers return extra fields on the assistant message — `reasoning`, `reasoning_content`, refusal blocks, annotations. Echoing them back re-sends the model's private scratchpad on every subsequent step: tokens you pay for, and a context that fills with stale thinking.

SRC/LLM.JS · *javascript*

```
export function sanitizeAssistant(message) {
  const out = { role: 'assistant', content: message.content ?? '' };
  if (Array.isArray(message.tool_calls) && message.tool_calls.length > 0) {
    out.tool_calls = message.tool_calls;
    if (message.content == null) delete out.content;
  }
  return out;
}
```

`max_tokens: 2048` is deliberately small: it keeps each step cheap and keeps whole files out of a single reply. The cost is that long answers get cut off — which is exactly what the truncation guard in the loop is for.

05 Messages are the memory

The `messages` array is the agent's entire working state. There is no database, no scratchpad file, no summary of what happened earlier. Delete the array and the agent has no idea it ever did anything.

ROLE	WHO WRITES IT	WHAT IT CARRIES
`system`	you	The job description: five numbered rules about looking before writing, copying snippets exactly, running what it wrote, and stopping when done.
`user`	you	The task — one plain sentence, or one corrective nudge (the truncation message is also a `user` turn).
`assistant`	the model	`content` (its thinking, printed as `think:`) and possibly `tool_calls` — its intentions.
`tool`	the harness	One string per call, tagged with the `tool_call_id` it answers. The only way the model learns what happened.

After three steps of the demo run, the array looks like this — JSON shortened, structure untouched:

THE ARRAY MID-RUN · *json*

```
[
  { "role": "system",    "content": "You are a small coding agent working inside a sandboxed folder. ..." },
  { "role": "user",      "content": "Show the loop: create a script, run it, edit it, run it again." },

  { "role": "assistant", "content": "Creating a script, then running it to check it works.",
    "tool_calls": [{ "id": "c1", "type": "function",
                     "function": { "name": "create_file", "arguments": "{\"path\":\"demo/hello.js\",...}" } }] },

  { "role": "tool", "tool_call_id": "c1", "content": "created demo/hello.js (72 bytes)" },

  { "role": "assistant", "content": "Now run it.",
    "tool_calls": [{ "id": "c2", "type": "function",
                     "function": { "name": "run_command", "arguments": "{\"command\":\"node demo/hello.js\"}" } }] },

  { "role": "tool", "tool_call_id": "c2", "content": "exit code 0\nstdout:\nHello from a tiny agent" }
]
```

This is also where the format gets strict. **Every `tool_call_id` must get a matching `tool` message**, or the next request is rejected by the provider as malformed. That is why the push happens inside the same `for` that ran the tool — even when the tool returned an `ERROR` string, because an error is still an answer.

> **Prompt design lives in three places, all of them ordinary code.** The system prompt in `prompt.js` (how to work), the schema descriptions in `tools.js` (what each tool is for), and the error strings the handlers return (how to recover). There is no separate "prompt file" to load at runtime.

The same array, reused for the next task

A task is one `runTask` call, and that is exactly where the step budget lives: the `for` starts at 1 every time, so the second task of a session opens at step 1/12 again rather than continuing a running total. What the session carries forward is this conversation, handed back in as `history`:

SRC/AGENT.JS · *javascript*

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },  // rebuilt for every task
  ...history.filter((m) => m.role !== 'system'),                    // the last task's turns
  { role: 'user', content: task },
];
```

Two details are the whole trick. The system message is *rebuilt* rather than carried, so a memory the previous task saved is already in the prompt the next task sends. And the cut has to land on a `user` message, because a provider rejects a conversation that starts with a `tool` result or ends with an unanswered `tool_calls`:

SRC/CLI.JS · *javascript*

```
export function trimHistory(messages, max = MAX_HISTORY) {
  const kept = (messages ?? []).filter((m) => m.role !== 'system');
  let start = Math.max(0, kept.length - Math.max(0, max));
  // Never start on a tool result or an unanswered tool_calls: providers reject it.
  while (start < kept.length && kept[start].role !== 'user') start += 1;
  return start >= kept.length ? [] : kept.slice(start);
}
```

That function is chapter 16's whole session: `/new` throws the array away, `/quit` ends the program, and `src/memory.md` outlives both.

> **One array, one run.** This memory dies with the process — a session carries it from task to task, but a restart still erases it. That is not a bug in the design, it is the boundary where chapter 12 begins: `memory.md` and the two extra tools that read and write it.

06 The eight tools

A tool is two things that must agree: a JSON schema the model reads, and a JavaScript function that runs when the model names it. The model never touches the filesystem — it emits JSON naming a tool, and the harness executes it. Six of the eight change the workspace; the last two change what the *next* process knows about it (chapter 12).

TOOL	WHAT IT DOES
`list_files`	Recursive listing, depth 3 by default (max 6), skipping `node_modules`, `.git`, `.venv`, `__pycache__`. Capped at 200 entries.
`read_file`	Line-numbered, paged via `offset` / `limit`, and says how many lines remain.
`create_file`	**Fails** if the file already exists — forces the model to edit rather than clobber.
`write_file`	Create or completely overwrite; parent folders are made for you.
`edit_file`	Exact-snippet replace. Zero matches is an error; more than one is an error unless `replace_all`.
`run_command`	Shell in the workspace root, 30 s timeout, returns exit code plus stdout and stderr.
`remember`	Append one durable note to `memory.md`: deduped, capped, dated. The only write that outlives the run.
`recall`	Search the whole memory file on demand — the lazy half of the pair.

The schema is the prompt

When the model chooses a tool it reads nothing but these descriptions. That makes them the highest-leverage text in the codebase — and the place to fix a model that keeps picking the wrong tool.

ONE SCHEMA FROM SRC/TOOLS.JS · *javascript*

```
{
  type: 'function',
  function: {
    name: 'create_file',
    description: 'Create a NEW file with the given content. Fails if the file already exists - use edit_file or
write_file for existing files.',
    parameters: {
      type: 'object',
      properties: {
        path:    { type: 'string', description: 'File path relative to the workspace root' },
        content: { type: 'string', description: 'Full file content' },
      },
      required: ['path', 'content'],
      additionalProperties: false,
    },
  },
}
```

Note what the description does: it states the *precondition* ("fails if the file already exists") and names the alternative. A model told that reaches for `edit_file` instead of blindly overwriting. `additionalProperties: false` matters too — it discourages invented arguments, and the dispatcher rejects wrong types anyway.

The dispatcher

SRC/TOOLS.JS · *javascript*

```
export async function runTool(name, rawArgs, ctx) {
  if (!HANDLERS[name]) {
    return `ERROR: unknown tool "${name}". Available: ${Object.keys(HANDLERS).join(', ')}`;
  }
  const { args, error } = validate(name, rawArgs);
  if (error) return error;

  try {
    return await HANDLERS[name](ctx.workspace, args, ctx);
  } catch (err) {
    if (err instanceof ForbiddenError) return `FORBIDDEN: ${err.message}`;
    if (err instanceof PathError) return `ERROR: ${err.message}`;
    return `ERROR: ${err.code ?? 'failed'}: ${err.message}`;
  }
}
```

Three rules are visible in those lines and they hold for every tool:

- **It returns a string, always.** Success, denial, unexpected exception — everything is text the model will read on the next step. A thrown error would escape the loop and end the run; a returned `ERROR: …` is an observation the model can act on.
- **Arguments are parsed and typed before dispatch.** `validate()` catches the common failure of a model sending `"limit": "20"` instead of `20`, and answers with `ERROR: "limit" must be a integer, got string` rather than a confusing crash two frames down.
- **Unknown names are survivable.** A hallucinated tool name gets a helpful list back and the loop keeps going. Models do this; the harness is expected to be patient about it.

> **Exit codes close the loop.** `run_command` reports `exit code 0` versus `exit code 1` as text, which is how the agent knows whether the code it just wrote actually works. A tool that returned only stdout would let a failing script look like a success.

07 edit_file, the interesting one

Seven of the eight tools are obvious. `edit_file` is the one that teaches something, because it is where an agent's output is most likely to be subtly wrong, and where good harness design turns that into a recoverable moment instead of a corrupted file.

SRC/TOOLS.JS (CONDENSED) · *javascript*

```
edit_file(ws, args) {
  const target = ws.resolve(args.path);
  if (!fs.existsSync(target.abs)) return `ERROR: no such file: ${target.rel}`;

  const oldStr = String(args.old_string);
  const newStr = String(args.new_string);
  if (oldStr === '') return 'ERROR: old_string must not be empty';

  const source = fs.readFileSync(target.abs, 'utf8');
  const count = source.split(oldStr).length - 1;

  if (count === 0) {
    return `ERROR: old_string not found in ${target.rel}. Read the file and copy the snippet exactly, including
whitespace.`;
  }
  if (count > 1 && !args.replace_all) {
    return `ERROR: old_string appears ${count} times in ${target.rel}. Add surrounding context to make it unique, or
pass replace_all: true.`;
  }

  const updated = args.replace_all
    ? source.split(oldStr).join(newStr)
    : source.replace(oldStr, () => newStr);   // function form: no $1 surprises
  fs.writeFileSync(target.abs, updated, 'utf8');

  return `edited ${target.rel}: ${args.replace_all ? count : 1} replacement(s), first at line ${lineNo}`;
}
```

Every failure mode, and why it is a message

SITUATION	WHAT COMES BACK	WHY THAT IS RIGHT
Snippet is not in the file	`ERROR: old_string not found … copy the snippet exactly, including whitespace.`	The model hallucinated whitespace. The message says what to do next, so the following step usually fixes it.
Snippet appears 4 times	`ERROR: old_string appears 4 times … pass replace_all: true`	Silent first-match replacement would edit the wrong function. Ambiguity must be an error, not a guess.
Snippet is `""`	`ERROR: old_string must not be empty`	An empty search string would otherwise "match" everywhere.
Replacement contains `$1`	replaced literally	`String.replace` interprets `$1` when the second argument is a string; the callback form removes the footgun.
File does not exist	`ERROR: no such file: …`	Points at the missing step without being asked.

This is the clearest demonstration of the fourth principle on the page: **an error is an observation, not a crash**. The failure costs one step and arrives as a sentence the model can read, which is why a live demo should include a forced failure — ask the agent to edit something in a way that cannot match, and watch the loop absorb it.

08 The sandbox

Two small files carry the safety story, and they answer different questions. `src/workspace.js` answers *does this path stay inside the root?* — the policy every file access goes through. `src/sandbox.js` answers the narrower, louder one: *may the agent change this file at all?* Reads are deliberately open, because an agent that cannot read a library, a log or a doc outside its own folder is useless; the fence is around `save`, `edit` and `delete`, because a read that leaves the sandbox is a bad idea and a write that leaves it is an incident.

SRC/SANDBOX.JS · *javascript*

```
export const SANDBOX_DIR = path.resolve(import.meta.dirname, '..', 'workspace');

check(action, requested) {                      // 'save' | 'edit' | 'delete'
  if (!ALLOWED.has(action)) throw new TypeError(`unknown sandbox action "${action}"`);

  let target;
  try {
    target = this.workspace.resolve(requested);  // workspace.js owns the path policy
  } catch (err) {
    if (err instanceof PathError) this.forbid(action, requested, err.message);
    throw err;
  }
  if (action === ACTIONS.DELETE && target.rel === '.')
    this.forbid(action, requested, 'the sandbox root itself cannot be deleted');
  return target;
}

forbid(action, requested, reason) {
  this.refusals += 1;
  this.log(ui.forbidden(action, requested, reason));   // the one bold red line
  throw new ForbiddenError(action, requested, reason);
}
```

A refusal is neither a stack trace nor a silent `undefined`. It prints one line in bold red capitals into the same console stream as the `[HOOK:Name]` lifecycle trace, then comes back to the model as an observation it can read and act on — so a model that tries the wrong path can correct itself on the next step:

TERMINAL · *console*

```
  ⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
result ← FORBIDDEN: SAVE ../escape.txt - path escapes the workspace: ../escape.txt
```

The policy it reuses is the one below. A plain `path.resolve` plus `startsWith` check is not enough on Windows, because several ordinary-looking strings resolve *inside* the workspace root:

SRC/WORKSPACE.JS · *javascript*

```
resolve(requested) {
  const raw = requested.trim();
  const expanded = expandVars(raw);                      // ~ and %VAR% and $VAR

  if (/^[A-Za-z]:/.test(raw))        throw new PathError(`absolute or drive-relative path is outside the workspace:
${raw}`);
  if (/^(\\\\|\/\/)/.test(raw))      throw new PathError(`UNC path is outside the workspace: ${raw}`);
  if (path.isAbsolute(expanded))     throw new PathError(`absolute path is outside the workspace: ${raw}`);

  const first = raw.split(/[\\/]/)[0].toLowerCase().replace(/:$/, '');
  if (/^[~$%]/.test(first) && first !== raw)
    throw new PathError(`home/environment path is outside the workspace: ${raw}`);
  if (HOME_AND_SYSTEM.has(first))
    throw new PathError(`"${first}" looks like a home/system folder, not a workspace file: ${raw}`);

  const abs  = path.resolve(this.root, expanded);
  const real = realResolve(abs);                         // deepest existing ancestor, realpath'd
  if (!isInside(this.root, real)) throw new PathError(`path escapes the workspace: ${raw}`);

  return { abs: real, rel: path.relative(this.root, real) };
}

resolveRead(requested) {                                 // reads: no containment check
  const expanded = expandVars(String(requested).trim());
  const real = realResolve(path.isAbsolute(expanded) ? expanded : path.resolve(this.root, expanded));
  return { abs: real, rel: path.relative(this.root, real) };
}
```

Every row below is refused by the write tools, before a byte is written, and the tests cover all of them:

THE MODEL ASKS FOR	WHAT HAPPENS	WHY A NAIVE CHECK FAILS
`C:\Windows\x.txt`	refused: absolute or drive-relative path	The regex fires before `path.resolve` can reinterpret `C:`.
`\\server\share\x`	refused: UNC path	UNC paths bypass drive-rooted containment entirely.
`~/Desktop/x.txt`	refused: home/environment path	`~` expands to the home directory after resolution.
`%USERPROFILE%\x`	refused: home/environment path	Environment expansion is not traversal, so `..` checks miss it.
`Desktop\notes.txt`	refused: looks like a home/system folder	With the cwd inside a user profile this resolves happily to a real Desktop.
`../../secrets.txt`	refused: path escapes the workspace	Caught late, by the containment check.
`link-to-home\x`	refused: path escapes the workspace	A junction would pass a string check; `realResolve` follows it first.

`realResolve` exists for the last row: it walks up to the deepest ancestor that exists, resolves *that* with `fs.realpathSync`, then re-appends the not-yet-created tail. A junction pointing out of the sandbox is compared by its real target, not by its name.

The same fence, over the shell

The file tools were never the whole story. `run_command` is a real shell, so `copy hello.js %USERPROFILE%\Desktop\hello.js` used to leave the sandbox without touching a single file tool. The command line says where it is writing, though, so `src/sandbox.js` reads it first and refuses the visible targets *before* the approval prompt:

SRC/SANDBOX.JS · *javascript*

```
const DEST_LAST     = new Set(['copy', 'cp', 'move', 'mv', 'xcopy', 'ren', 'scp', 'rsync']);
const ALL_POSITIONAL = new Set(['del', 'erase', 'rd', 'rmdir', 'rm', 'md', 'mkdir', 'touch', 'tee']);
const OUTPUT_FLAG   = { curl: ['-o', '--output', '-O'], wget: ['-O', '--output-document'] };

checkCommand(command) {
  const targets = shellWriteTargets(command);      // tokens → words and operators
  const refused = [];

  for (const { verb, target } of targets) {
    try {
      this.workspace.resolve(target);              // the same policy as a write_file
    } catch (err) {
      if (!(err instanceof PathError)) throw err;
      this.refusals += 1;
      this.log(ui.forbidden(verb, target, err.message));
      refused.push(`${verb.toUpperCase()} ${target}`);
    }
  }

  if (refused.length)
    throw new ForbiddenError('command', refused.join(', '), 'the shell would write outside the sandbox');
  return { ok: true, targets };
}
```

COMMAND	WRITE TARGET IT IS CHECKED FOR
`copy a.txt C:\Users\me\Desktop\a.txt`	last positional argument — `C:\Users\me\Desktop\a.txt`
`robocopy src C:\Backup /MIR`	the second positional argument
`del /q old.txt`, `rm -rf build`	every non-switch operand
`echo hi > out.txt`, `node build.js 2> errors.log`	the redirection target (a bare digit before `>` is a stream id)
`curl -o C:\Temp\x.zip https://example.com/x.zip`	the argument after `-o`
`Copy-Item a.txt -Destination C:\Temp\`	the argument after `-Destination`, `-Path`, `-LiteralPath`

> **The honest limit.** The scan reads the command line, not the code inside a script. `node -e "fs.copyFileSync('a','C:/Users/me/Desktop/a')"`, a Python one-liner or a downloaded script can still write anywhere. This is a seatbelt around the obvious commands, not an operating-system sandbox — isolation is a container's job, and saying so out loud beats implying the demo is safe by construction.

The refusal console line is the same one the file tools print, so on stage there is exactly one thing to point at: `FORBIDDEN`, in bold red capitals, with the action and the path that caused it.

09 Human in the loop

Everything so far lets a model act. This chapter is the part that lets *you* decide how much. There is one setting, `CMD_POLICY`, and it governs the only tool that can do damage the file fence cannot see: `run_command`. Even before the policy is consulted, the command's visible write targets are checked against the sandbox (chapter 08), so a `copy` to the Desktop never reaches the approval prompt.

`CMD_POLICY`	BEHAVIOUR	WHEN YOU WOULD USE IT
`ask` default	Every command stops and waits for a `y/N` at the terminal. `n` returns `DENIED: the user did not approve this command.` to the model.	Live demos, and any run you are watching. The pause is the point.
`allow`	Commands run without asking. Set by `--yes` / `--allow-commands`, or by `CMD_POLICY=allow`.	The scripted demo, CI, or a task you already trust.
`deny`	No command runs at all; the model gets a denial and can still edit files.	Read-mostly tasks, or a run with no terminal attached.

The approval sits in the handler, not in the loop, so a denied command behaves like any other observation:

SRC/TOOLS.JS (CONDENSED) · *javascript*

```
async run_command(ws, args, ctx) {
  const command = String(args.command ?? '').trim();
  if (!command) return 'ERROR: command is required';

  for (const { re, why } of HARD_DENY) {
    if (re.test(command)) return `DENIED: blocked by policy (${why}). Command: ${command}`;
  }

  sandboxFor(ctx).checkCommand(command);   // refuses copy/move/del/redirects outside the workspace

  const policy = ctx.commandPolicy;
  if (policy === 'deny') return 'DENIED: command execution is disabled (CMD_POLICY=deny).';

  const approval = policy === 'allow' ? 'allow' : await ctx.confirm(command);
  if (approval !== 'allow') return `DENIED: the user did not approve this command. Command: ${command}`;

  const { code, stdout, stderr } = await runShell(command, ws.root);
  ...
}
```

What is refused no matter what the policy says

Eight patterns are checked before the policy is consulted, so `allow` cannot unblock them. They are not a security boundary — they are a seatbelt against a model that has read too many bad shell examples.

PATTERN	REFUSED BECAUSE
`rm -rf /`, `rm -rf ~`	recursive delete of a root path
`del /s`, `del /q`	recursive or quiet Windows delete
`format`, `diskpart`	disk formatting
`shutdown`, `reboot`	machine power control
`git push --force`, `git push -f`	force push
`curl … | sh`, `wget … | bash`	piping the internet into a shell

PATTERN	REFUSED BECAUSE
`:(){ :|:& };:`	fork bomb
`> /dev/sda`	raw disk write

The non-TTY rule. With no terminal attached (piped input, CI, a scheduled run) the `ask` policy degrades to **deny**. An unanswerable prompt must never become an approval — the `confirm` function defaults to `async () => 'deny'` for exactly this reason.

10 The console palette

One file knows about colour. Every line the agent prints is a named helper in `src/ui.js`, so the loop itself contains no escape codes and the whole palette can be swapped in one place.

SRC/UI.JS (CONDENSED) · *javascript*

```
export const ui = {
  title:   (text) => chalk.bold.cyan(text),
  field:   (label) => `${chalk.green(label)} ${chalk.white('›')}`,
  kv:      (label, value) => `${chalk.blue(`${label.padEnd(10)}:`)} ${chalk.white(value)}`,
  step:    (step, max, width = 40) => chalk.blue(`── step ${step}/${max} ${'─'.repeat(width)}`),
  think:   (text) => `${chalk.cyan('think:')} ${text}`,
  tool:    (name, args) => `${chalk.white('tool →')} ${chalk.yellow(name)} ${chalk.white(args)}`,
  result:  (text) => isFailure(firstLine(text))
             ? chalk.red(`result ← ${firstLine(text)}`)
             : `${chalk.green('result ←')} ${chalk.white(firstLine(text))}`,
  answerRule: () => chalk.bold.green(`── final answer ${'─'.repeat(38)}`),
  answer:  (text) => chalk.white(text),
  hookLine:(icon, name, colour, when) => `  ${icon}  ${chalk[colour].bold(name.padEnd(16))} ${when}`,
  hookField:(label, value) => `      ${chalk.dim(label.padEnd(9))} ${value}`,
  hookTrace:(icon, name, colour, detail) => `  ${icon}  ${chalk[colour](name.padEnd(16))} ${chalk.dim(detail)}`,
  warn:    (text) => chalk.yellowBright(text),
  waiting: (text) => chalk.yellowBright(text),
  error:   (text) => chalk.red(text),
  note:    (text) => chalk.white(text),
  prompt:  (text) => chalk.green(text),
  dim:     (text) => chalk.dim(text),
};
```

Two design decisions make this more than cosmetics.

- **chalk styles the exact string you hand it.** There is no markup to parse, so text from the model or a tool prints verbatim: a file containing `<red>` shows up as `<red>`, and Windows backslashes and JSON survive intact. A console library that interprets inline tags would quietly corrupt the very output the audience is reading.
- **Every helper returns a string and nothing writes to stdout.** That is why `runTask` can take an injected `log`, and why the loop can be tested without capturing a terminal.

The lifecycle hooks get their own group of helpers here too — `hookLine`, `hookField`, `hookTrace`, `hookFlow`. The hook names, icons and colour *names* live in `hooks.js` as plain data; the colour itself is still painted in this file, which is why chapter 11 can describe nine hooks without ever mentioning an escape code.

The colour of a result is derived, not reported by the tool: `isFailure` looks for `ERROR`, `DENIED` or an exit code other than 0 in the first line.

SRC/UI.JS · *javascript*

```
const isFailure = (text) => /^(ERROR|DENIED)\b/.test(text) || /\bexit code (?!0\b)\d+/.test(text);
```

That small regex is the visual contract of the whole demo: **anything red means the model has a problem to solve on the next step**. chalk decides whether to colour at all — `NO_COLOR`, `FORCE_COLOR` and "output is a pipe" are all honoured — so redirecting a run to a file gives clean text.

11 Lifecycle hooks

The loop is fixed on purpose — that is what makes it readable — but a fixed loop is not *extensible*. "Log every command", "never let it touch `.env`", "tell me when it gives up": every one of those used to mean editing `agent.js`, and every one of those edits is a chance to break the loop for everybody. `src/hooks.js` is the smallest thing that fixes that. A **hook** is a function attached to a named moment in the loop; the loop calls all nine moments, and your code never edits the loop at all.

Two things make this a chapter rather than a footnote. First, the nine moments are the shape of every production harness you will meet — permissions, audit logs, redaction and compaction are all the same mechanism. Second, it is the honest answer to the question an audience always asks at the end: *"so how would you make this real?"*

The nine moments

#	HOOK	ICON	COLOUR	FIRES	WHAT A HOOK MAY DO
1	`SessionStart`	🚀	cyan	once, before the first model call	**refuse to start** - or post a note
2	`UserPromptSubmit`	✍️	blue	once, as the task enters the loop	**rewrite the task**, or block the run
3	`PreModel`	🧠	magenta	before every model call	observe: the request is assembled, not sent
4	`PostModel`	💬	cyan	after every reply, before dispatch	observe: answer or tool call, visible first
5	`PreToolUse`	🛠️	bright yellow	before every tool call	**block** - the tool never runs
6	`PostToolUse`	📦	green	after every tool call	**replace** the observation the model reads
7	`Stop`	🏁	bright green	the model answered and the loop is ending	observe: summary, cost, notification
8	`StepLimit`	⏳	yellow	the loop hit `maxSteps`	observe: escalate, dump the transcript
9	`SessionEnd`	🔒	red	always - finished or not	observe: the last code to run in the process

ONE RUN, IN ORDER · *text*

```
SessionStart
  └─ UserPromptSubmit
       └─ ┌ PreModel ─▶ model call ─▶ PostModel ─▶ PreToolUse ─▶ tool ─▶ PostToolUse ┐
          └──────────────────── each step repeats this ──────────────────────────────┘
                                  │
                        Stop (an answer)  or  StepLimit (the budget)
                                  │
                             SessionEnd  ← always, whichever way it went
```

Seeing it, before reading it

The trace is on by default: every run - `npm start` as much as `npm run demo` - prints each lifecycle moment as one line, tagged `[HOOK:Name]`, followed by what that moment is *for* and what this firing carried. It needs no key and no network, and one flag explains it:

TERMINAL · *bash*

```
node src/cli.js --hooks             # every hook: icon, colour, what it gets, why it exists, then exit
node src/cli.js --demo              # the same lifecycle, printed as each moment fires
node src/cli.js --demo --no-hooks   # the plain loop, with the trace silenced
```

`--hooks` prints the catalogue. It is generated from `HOOK_EVENTS`, the same array the loop fires, so this page and the console cannot drift apart:

NODE SRC/CLI.JS --HOOKS · *console*

```
── lifecycle hooks - 9 moments in one run ──────────────────────
  A hook is a function attached to a named moment in the loop. agent.js calls each
  moment; a hook may observe, rewrite what the model reads, or block a call outright.

  🛠  PreToolUse       before every tool call
      receives  name, args, step
      may       BLOCK - return { block: true, reason } and the tool never runs
      The one hook that changes what the agent is allowed to do. A permission layer
      lives here: inspect the arguments, refuse the call. The refusal comes back as an
      observation, so the model can choose another way instead of crashing.

── the order one run fires them ────────────────────────────────
  1. SessionStart
  2. UserPromptSubmit
  3. [ PreModel → PostModel → PreToolUse → PostToolUse ]
  4. Stop | StepLimit
  5. SessionEnd

  9 hooks, and 4 of them can change the run: SessionStart (refuse to start the run);
  UserPromptSubmit (rewrite or block the task); PreToolUse (block a tool call);
  PostToolUse (replace the observation). The rest only watch - which is exactly why a
  hook is a safe first extension point.
```

The default trace is one observer per event, attached by `traceHooks()`. This is a real excerpt from `node src/cli.js --demo`; each tag is painted in its hook's own colour:

THE LIFECYCLE, LIVE · *console*

```
  🚀 [HOOK:SessionStart] once, before the first model call - Show the loop, then try to break the sandbox: create a
script, run it, e
  ✍️ [HOOK:UserPromptSubmit] once, as the task enters the loop - Show the loop, then try to break the sandbox: create
a script, run it, e

── step 3/12 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 3/12 - 6 messages
  💭 [HOOK:PostModel] after every reply, before anything is dispatched - step 3 - tool_calls
think: Now run it - a real shell, a real exit code.
tool → run_command {"command":"node demo/hello.js"}
  🛠  [HOOK:PreToolUse] before every tool call - run_command {"command":"node demo/hello.js"}
result ← exit code 0
        ↳ stdout: Hello from a tiny agent
  📦 [HOOK:PostToolUse] after every tool call - run_command - exit code 0
...
  🏁 [HOOK:Stop] the model answered and the loop is ending - 11 step(s) - Done. Created demo/hello.js, ran it, edited
one string,
  🔒 [HOOK:SessionEnd] always - every run ends here, finished or not - ok (answer) in 498ms
```

Read the trace next to chapter 03's loop and the picture is complete: the hooks sit exactly where the `hook.run(...)` lines are, in the same order, on the same iteration. Nothing is interleaved by magic.

The contract

A hook is called with a payload object and its *return value* decides what happens. That is the whole API, and it is deliberately three verbs wide:

THE WHOLE PROTOCOL · *javascript*

```
return undefined;                              // observe only - the run is untouched
return { note: 'checked' };                    // say something; still untouched
return { task: '…and run the tests' };         // UserPromptSubmit: rewrite the task
return { replace: 'sk-or-[redacted]' };        // PostToolUse: change what the model reads
return { block: true, reason: 'no' };          // SessionStart/UserPromptSubmit/PreToolUse: refuse
```

Two consequences are worth stating plainly, because they are the difference between a hook system that survives contact with a real team and one that does not:

- **A hook that throws is reported and ignored.** One broken hook must not take the loop with it: the handler is caught, printed as `hook PostToolUse (…') threw: … - ignored`, and the next handler runs. Above all, every `tool_call_id` still gets a matching tool message, or the next request would be invalid.
- **A refusal is an observation, not an exception.** When `PreToolUse` blocks a call, the tool is never dispatched *and* the model is told why: `DENIED: a PreToolUse hook blocked this call (no pushes from the sandbox). Try another approach.` The loop cannot crash on a "no", and the model gets its next turn to choose differently — which is principle 4, applied to permissions.

The code

The registry is an ordinary object with a map in it. `on` attaches a function to an event name (a typo throws instead of silently never firing), `off` detaches it, and `run` calls the handlers in order and collects their decisions into one answer:

SRC/HOOKS.JS · *javascript*

```
export class Hooks {
  async run(name, payload = {}) {
    const out = { event: name, ran: 0, blocked: false, reason: null, task: null, replace: null, notes: [], errors: []
};

    for (const handler of [...this.#handlers.get(name)]) {
      if (handler.once) this.off(name, handler.id);
      out.ran += 1;

      let decision;
      try {
        decision = await handler.fn(payload, { event: name });
      } catch (err) {
        out.errors.push(`${handler.id}: ${err.message}`);
        this.log?.(ui.warn(`  hook ${name} (${handler.id}) threw: ${err.message} - ignored`));
        continue;                                  // one bad hook is not the loop's problem
      }

      if (!decision || typeof decision !== 'object') continue;   // observe-only
      if (decision.note)    out.notes.push(String(decision.note));
      if (typeof decision.task    === 'string') out.task = decision.task;
      if (typeof decision.replace === 'string') out.replace = decision.replace;

      if (decision.block) {                        // first refusal wins, rest are skipped
        out.blocked = true;
        out.reason = decision.reason ? String(decision.reason) : `${name} blocked this step`;
        break;
      }
    }
    return out;
  }
}
```

The loop side is one line per moment, which is the point — the extensibility costs nine lines in `runTask` and nothing else in the program:

AGENT.JS, THE PERMISSION LAYER IN THREE LINES · *javascript*

```
const pre = await hook.run('PreToolUse', { name, args: rawArgs, step });
const result = pre.blocked
  ? `DENIED: a PreToolUse hook blocked this call (${pre.reason}). Try another approach.`
  : await runTool(name, rawArgs, ctx);
```

A permission layer and a redactor, in twelve lines

This is a real, runnable use of the API — it is what the test suite exercises:

YOUR CODE, NOT MINE · *javascript*

```
import { runTask } from './agent.js';
import { Hooks, printHookCatalogue } from './hooks.js';

const hooks = new Hooks({ log: console.log });        // where broken-hook warnings go

// 1. A permission layer. The model asks; the hook decides.
hooks.on('PreToolUse', ({ name, args }) => (
  name === 'run_command' && /\bgit push\b/.test(args)
    ? { block: true, reason: 'no pushes from the sandbox' }
    : undefined
));

// 2. Redaction. Whatever a command printed, a key never reaches the model.
hooks.on('PostToolUse', ({ result }) => ({
  replace: String(result).replace(/sk-or-\S+/g, 'sk-or-[redacted]'),
}));

// 3. Failures get a lifecycle too.
hooks.on('SessionEnd', ({ ok, reason, durationMs }) => ({
  note: `run ${ok ? 'ok' : `failed (${reason})`} in ${durationMs}ms`,
}));

printHookCatalogue({ hooks });                        // marks the ones that are attached
await runTask({ task, workspace, commandPolicy, maxSteps, chatFn, hooks });
```

With that attached, a `git push origin main` never runs, and the model is told so as a tool result:

WHAT THE MODEL SEES · *console*

```
tool → run_command {"command":"git push origin main"}
result ← DENIED: a PreToolUse hook blocked this call (no pushes from the sandbox). Try another approach.
```

> **A tool is how the model acts on the world; a hook is how you act on the loop.** Tools are the model's hands and they are described in the prompt, so the model chooses them. Hooks are yours: they are not described to the model, they get no vote from it, and they run whether it likes it or not. That asymmetry is the whole safety story of a real harness — and here it is nine lines, in a file you can read on a projector.
> **Why nine, and why these nine?** Because the nine are the places the loop *already* pauses: before it starts, before the first request, before and after each reply, before and after each tool call, on success, on exhaustion, and at the end. A hook system with an event for something the loop does not do is documentation, not an extension point. If you add a tenth, add it where the code pauses.

12 Long-term memory

Everything so far lives and dies inside one process. `messages` is the agent's working memory, and when the process exits it is gone — including the lessons that were expensive to learn ("the tests are offline", "this repo spells it Brighton", "force-push is refused"). Chapter 11 gave you a place to hang code on the loop. This chapter adds the smallest thing that actually persists: a markdown file, and two tools that read and write it.

The store is `src/memory.md` — beside the code, not inside the sandbox, so you can open it in your editor and the agent can only reach it through `remember`. One bullet per memory, and the bullet *is* the record — no database, no index, no schema migration, nothing to migrate when you change your mind.

SRC/MEMORY.MD · *markdown*

```
# memory

Long-term notes this agent keeps between runs. One bullet per memory:
`- [tag] text _(YYYY-MM-DD)_` where tag is fact | pref | fix | gotcha.

This file IS the store - edit it by hand, delete a wrong line, or commit it.
The agent reads a bounded slice at the start of a run and can search the rest
with the `recall` tool.

- [fact] The sandbox demo script lives at demo/hello.js and is safe to regenerate with create_file. _(2026-10-28)_
```

Three consequences, all of them the point:

- **No cache, no drift.** Every method reads the file. A hand edit, a deletion or a `git checkout` is picked up on the next call — there is no state that can disagree with the text on disk. A memory module with a cache is a memory module that lies.
- **It diffs like code.** `src/memory.md` is a normal file in the repo: it appears in `git status`, it diffs cleanly, and a memory can be reviewed or deleted in a pull request. Point `MEMORY_FILE` at an absolute path to share one store across checkouts.
- **You can audit it.** `node src/cli.js --memory` prints the file and the exact block the prompt received. "What does it remember about me?" is a `cat`, not an interrogation.

Reading it: eager, then lazy

The two access paths cost different things, so the harness has both. `runTask` reads the file once and pastes a *bounded* slice into the system prompt; anything that did not fit is still one tool call away.

PATH	WHAT IT IS	WHAT IT COSTS
eager — `digest()`	The newest 12 entries, capped at 1200 characters, appended to the system prompt.	A fixed number of characters. It cannot grow with the file, so it cannot quietly eat the context window.
lazy — `recall`	A substring search over the whole file, when the model asks for it.	One tool call, and only when the task needs it.

The digest always ends by saying what it left out. That is the difference between "this is all I know" and "this is what fits — ask for the rest", and it is one line of code in `memory.js`:

WHAT THE SYSTEM PROMPT ACTUALLY RECEIVES · *text*

```
## What you remember from earlier runs
These notes live in memory.md and were learned before this task started.
Treat them as strong hints, not facts to trust blindly: the code may have moved on since.
- [gotcha] create_file refuses to overwrite, so re-running the demo fails at step 2. _(2026-10-24)_
- [fix] The demo script lives at demo/hello.js; delete it before a clean demo run. _(2026-10-28)_
- [pref] The user wants console output, not files, for anything under three lines. _(2026-10-28)_
(9 older memories not shown - search memory.md with the recall tool)
```

And the whole integration in the loop is four lines at the top of `runTask` — no new message role, no special channel, no vector store. Memory is text in the one message the model always receives:

SRC/AGENT.JS · *javascript*

```
const messages = [
  { role: 'system', content: buildSystemPrompt({ memory, log }) },
  { role: 'user', content: task },
];
const ctx = { workspace, commandPolicy, memory, confirm };
```

...and the two lines that do the work are in `prompt.js`, because "what the model is told" is one decision made in one place: the rules, then the digest, then a one-line note to the console saying how many memories were carried.

SRC/PROMPT.JS · *javascript*

```
// buildSystemPrompt({ memory, log }) - the whole of it
const digest = memory.digest();
if (!digest) return rules;
return `${rules}\n\n${digest}`;
```

The code

Parsing is one regex, and anything that is not a tagged bullet is ignored — which is what makes prose, headings and comments safe in a file a human is meant to edit.

SRC/MEMORY.JS · *javascript*

```
export function parseMemory(source) {
  const entries = [];
  for (const line of String(source ?? '').split(/\r?\n/)) {
    // - [tag] text _(YYYY-MM-DD)_   (the date is optional for hand-written bullets)
    const m = line.match(
      /^\s*[-*]\s*\[([a-z]+)\]\s*(.+?)\s*(?:_\((\d{4}-\d{2}-\d{2})\)_)?\s*$/i,
    );
    if (!m) continue;
    const text = normalise(m[2]);
    if (!text) continue;
    entries.push({ tag: m[1].toLowerCase(), text, date: m[3] ?? 'undated' });
  }
  return entries;
}
```

A write is where the bounds live. Duplicates are refreshed rather than appended (saying the same thing again is not new information), and the oldest entries are dropped so the file cannot grow without limit — the same argument as `maxSteps`, one level up.

SRC/MEMORY.JS · *javascript*

```
remember(rawText, { tag = 'fact' } = {}) {
  const text = normalise(rawText).slice(0, this.options.rememberChars);
  const entry = {
    tag,
    text: text.slice(0, this.options.maxEntryChars),
    date: today(this.now()),
  };

  const entries = this.entries();
  const existing = entries.findIndex(
    (e) => e.text.toLowerCase() === text.toLowerCase(),
  );
  if (existing !== -1) {          // the same note again: refresh the date
    entries[existing] = { ...entries[existing], date: entry.date, tag: entry.tag };
    this.save(entries);
    return { ok: true, duplicate: true, entry: entries[existing], total: entries.length };
  }

  entries.push(entry);
  // Bounded on purpose: the file cannot grow without limit, like maxSteps.
  const dropped = Math.max(0, entries.length - this.options.maxEntries);
  if (dropped) entries.splice(0, dropped);
  this.save(entries);
  return { ok: true, entry, dropped, total: entries.length };
}
```

Writing it: one tool call, not one per turn

`remember` is a normal tool, so a memory write appears in the transcript like everything else. Nothing writes automatically at the end of a run: a model that saved a memory every step would be filling your context with its own diary, and the system prompt says so in one line — durable facts only, never task progress.

NPM RUN DEMO, STEPS 11 AND 12 · *text*

```
── step 11/14 ──────────────────────────────────────
think: Saving one durable fact, the way a later run would find it again:
tool → remember {"text":"The sandbox demo script lives at demo/hello.js and is safe to regenerate with
create_file.","tag":"fact"}
result ← remembered in src/memory.md (3 total): - [fact] The sandbox demo script lives at demo/hello.js and is safe to
regenerate with create_file. _(2026-10-28)_

── step 12/14 ──────────────────────────────────────
think: Reading memory back the lazy way - the same file the system prompt was built from:
tool → recall {"query":"demo"}
result ← 1 of 3 memories in src/memory.md: - [fact] The sandbox demo script lives at demo/hello.js and is safe to
regenerate with create_file. _(2026-10-28)_
```

Run the demo a second time and the harness says what it carried in, before step 1 — the write from the previous process is now context, not history:

SECOND RUN, SAME WORKSPACE · *text*

```
memory    : src/memory.md (2 memories, newest 2026-10-28)

memory ← src/memory.md - 2 memories added to the system prompt
```

The caps

Every number below exists to keep memory from becoming a leak with a friendly name:

KNOB	DEFAULT	WHAT IT BOUNDS
`MEMORY_MAX_ENTRIES`	60	the file — the oldest entries are dropped when it is reached
`MEMORY_PROMPT_CHARS`	1200	the injected block; `0` turns injection off and leaves `recall` working
`maxEntryChars`	300	one memory, so no single note can dominate
`rememberChars`	2000	one `remember` call
`promptEntries`	12	how many of the newest appear in the digest

That last combination is the interesting one: `MEMORY_PROMPT_CHARS=0` turns the harness from "stuff what fits" into "retrieve when needed", with no code change at all.

> **What this is not.** No embeddings, no vector search, no summarising, no write on every turn. It is one file a human can read, diff and delete a line from — which is why the answer to "why did it remember that?" is `node src/cli.js --memory` rather than a log hunt. Every real system grows past this; almost all of them start here.

13 Configuration and precedence

Three sources compete for the key and the model name, and the winner is printed on every start with its provenance. The ladder is deliberately *not* the usual one:

PRECEDENCE · *text*

```
--model  flag          ← highest
.env  (nearest file)  ← beats the shell, on purpose
exported environment  ← lowest for anything .env defines
```

`.env` beating the shell is a choice, not an accident: a stale `OPENROUTER_API_KEY` in `~/.bashrc` would otherwise silently outrank the key you just wrote, and you would spend an afternoon on a 401. Variables `.env` does not mention still come from the environment.

Finding the file is a walk upwards, so the single `.env` at the repository root works for a package one level down:

SRC/CONFIG.JS · *javascript*

```
export function findEnvFile(startDir = PROJECT_ROOT) {
  let dir = path.resolve(startDir);
  for (;;) {
    const candidate = path.join(dir, '.env');
    if (fs.existsSync(candidate) && fs.statSync(candidate).isFile()) return candidate;
    const parent = path.dirname(dir);
    if (parent === dir) return null;      // filesystem root: stop
    dir = parent;
  }
}
```

The knobs

VARIABLE	DEFAULT	WHAT IT DOES
`OPENROUTER_API_KEY` / `OPENROUTER_KEY`	required	The key. Only ever printed as `sk-or...aaa5 (73 chars)`.
`OPENROUTER_MODEL` / `MODEL`	`deepseek/deepseek-v4.1-flash-1017`	Which model. `MODEL=` is an alias for older `.env` files in this repo.
`CMD_POLICY`	`ask`	`allow` / `ask` / `deny`.
`MAX_STEPS`	`12`	How many model calls the loop may make.
`REQUEST_TIMEOUT_MS`	`90000`	Per-call transport timeout.
`WORKSPACE_DIR`	`workspace`	The sandbox root, resolved against the package.
`MEMORY_FILE`	`src/memory.md`	The long-term memory file, resolved against the package root (an absolute path points at a shared store).
`MEMORY_MAX_ENTRIES`	`60`	Hard cap on memories; the oldest are dropped at the limit.
`MEMORY_PROMPT_CHARS`	`1200`	Budget for the injected memory block; `0` disables injection and leaves `recall` working.

Two small kindnesses

- **Fingerprinting instead of printing.** `fingerprint()` returns `sk-or...aaa5 (73 chars)` — enough to tell two keys apart on stage, useless in a screenshot.
- **Normalising bare model names.** OpenRouter needs `vendor/model`, and a `.env` carried over from another project usually holds `gpt-5`. A prefix table maps `deepseek*`, `claude*`, `gpt*`, `gemini*`, `grok*`, `llama*`, `qwen*`, `mistral*` to a vendor, and the banner says when it guessed.

SRC/CONFIG.JS · *javascript*

```
const VENDOR_BY_PREFIX = [
  [/^deepseek/i, 'deepseek'],   [/^claude/i, 'anthropic'],   [/^(gpt|o[13458]|chatgpt)/i, 'openai'],
  [/^gemini/i, 'google'],       [/^grok/i, 'x-ai'],           [/^llama/i, 'meta-llama'],
  [/^qwen/i, 'qwen'],           [/^mistral|^mixtral/i, 'mistralai'],
];
```

14 Bounded autonomy

An agent that can loop is an agent that can loop forever. Four bounds cap it, and each one has a visible consequence rather than a silent stall.

BOUND	WHAT IT PREVENTS	WHAT YOU SEE
`maxSteps` (12)	A model that keeps reading files and never concludes.	`stopped: hit the 12-step limit`, exit code 3.
`finish_reason: length` guard	Half a tool call, dispatched as JSON that will not parse.	`reply was truncated - asking the model to continue smaller`.
`REQUEST_TIMEOUT_MS` (90 s)	A socket that never answers.	`request failed: request timed out after 90s`, exit code 1.
`run_command` timeout (30 s)	A command that hangs the whole run.	`[tool] command timed out after 30s` in the stderr block.

The heartbeat

Between them, those bounds would still leave a 90-second silence, which on a projector looks identical to a crash. So a timer prints something every five seconds while the request is in flight:

SRC/HEARTBEAT.JS · *javascript*

```
export function heartbeat(log, everyMs = 5000) {
  const started = Date.now();

  const timer = setInterval(() => {
    const seconds = Math.round((Date.now() - started) / 1000);
    log(ui.waiting(`  … still waiting for the model (${seconds}s)`));
  }, everyMs);

  timer.unref?.();                  // never keep the process alive on its own
  return { stop: () => clearInterval(timer) };
}
```

It is armed just before the call and stopped just after, including on the failure path. `unref()` matters: a timer that holds the event loop open would turn a clean exit into a program that hangs for five more seconds.

Exit codes

CODE	MEANING
0	The model stopped calling tools and gave an answer.
1	The request failed — bad key, no credits, timeout, non-JSON.
2	No task: nothing typed at the prompt, the field closed, or stdin is not a terminal.
3	Stopped at the step limit.

Why the truncation guard is the interesting one. A cut-off reply often ends mid-JSON: `{"path": "hello.js", "cont`. Dispatching that would produce `ERROR: could not parse arguments` for a tool call the model never finished, and the model would spend a step apologising for a bug it did not have. Refusing to dispatch it is the difference between a harness that respects the model and one that argues with it.

15 A real run, annotated

Everything on this page so far is description. This is a real run, captured from `node src/cli.js --demo` with colour disabled, unedited:

TERMINAL · *console*

```
min-coding-js
workspace : C:\Users\mrcra\Desktop\BrightonDev.ai-meetup-nov-2026\mini-coding-agent-js\workspace
sandbox   : save, edit, delete - only inside C:\Users\mrcra\Desktop\BrightonDev.ai-meetup-nov-2026\mini-coding-agent-
js\workspace (reads: anywhere)
model     : scripted (no network)
key       : -
env file  : C:\Users\mrcra\Desktop\BrightonDev.ai-meetup-nov-2026\.env
commands  : allow
timeout   : 90s per model call
tools     : list_files, read_file, create_file, write_file, edit_file, run_command, remember, recall
memory    : src/memory.md (11 memories, newest 2026-10-30)
hooks     : on: each of the 9 prints as [HOOK:name] - see --hooks, silence with --no-hooks

task      : Show the loop, then try to break the sandbox: create a script, run it, edit it, run it again, read a file
from outside the workspace, then attempt a path escape, a home-folder write, a force push and an API-key echo.

  9 lifecycle hooks attached - each prints as it fires, e.g.
  🛠️  [HOOK:PreToolUse] before every tool call - run_command {"command":"ls"}
  (--no-hooks hides these; --hooks explains them)

memory ← src/memory.md - 5 memories added to the system prompt
  🚀 [HOOK:SessionStart] once, before the first model call - Show the loop, then try to break the sandbox: create a
script, run it, e
  ✍️  [HOOK:UserPromptSubmit] once, as the task enters the loop - Show the loop, then try to break the sandbox: create
a script, run it, e

── step 1/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 1/16 - 2 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 1 - tool_calls
think: Looking first: what is in the workspace, and what does the sandbox allow.
tool → list_files {"path":"."}
  🛠️  [HOOK:PreToolUse] before every tool call - list_files {"path":"."}
result ← .gitkeep  (71b)
  📦 [HOOK:PostToolUse] after every tool call - list_files - .gitkeep  (71b)

── step 2/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 2/16 - 4 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 2 - tool_calls
think: Creating a script, then running it to check it works.
tool → create_file {"path":"demo/hello.js","content":"const agent = 'tiny agent';\nconsole.log(`Hello from a
${agent}`);\n"}
  🛠️  [HOOK:PreToolUse] before every tool call - create_file {"path":"demo/hello.js","content":"const agent = 'tiny
agent
result ← ERROR: demo/hello.js already exists - use edit_file to change it or write_file to overwrite
  📦 [HOOK:PostToolUse] after every tool call - create_file - ERROR: demo/hello.js already exists - use edit_file to
chang

── step 3/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 3/16 - 6 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 3 - tool_calls
think: Now run it - a real shell, a real exit code.
tool → run_command {"command":"node demo/hello.js"}
  🛠️  [HOOK:PreToolUse] before every tool call - run_command {"command":"node demo/hello.js"}
result ← exit code 0
         ↳ stdout: Hello from a tiny agent, edited by a tool call, edited by a tool call, edited by a tool call, edited
by a tool call
  📦 [HOOK:PostToolUse] after every tool call - run_command - exit code 0

── step 4/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 4/16 - 8 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 4 - tool_calls
think: Fixing the copy in the script.
```

```
tool → edit_file {"path":"demo/hello.js","old_string":"Hello from a ${agent}","new_string":"Hello from a ${agent}, edited by a tool call"}
  🛠️  [HOOK:PreToolUse] before every tool call - edit_file {"path":"demo/hello.js","old_string":"Hello from a ${agent}"
result ← edited demo/hello.js: 1 replacement(s), first at line 2
  📦 [HOOK:PostToolUse] after every tool call - edit_file - edited demo/hello.js: 1 replacement(s), first at line 2

── step 5/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 5/16 - 10 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 5 - tool_calls
think: Re-running to verify the edit.
tool → run_command {"command":"node demo/hello.js"}
  🛠️  [HOOK:PreToolUse] before every tool call - run_command {"command":"node demo/hello.js"}
result ← exit code 0
         ↳ stdout: Hello from a tiny agent, edited by a tool call, edited by a tool call, edited by a tool call, edited by a tool call, edited by a tool call
  📦 [HOOK:PostToolUse] after every tool call - run_command - exit code 0

── step 6/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 6/16 - 12 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 6 - tool_calls
think: Now the fence. First a write that tries to climb out of the workspace:
tool → write_file {"path":"../escape.txt","content":"this file must never be written"}
  🛠️  [HOOK:PreToolUse] before every tool call - write_file {"path":"../escape.txt","content":"this file must never be w
  ⛔ FORBIDDEN SAVE ../escape.txt - path escapes the workspace: ../escape.txt
result ← FORBIDDEN: SAVE ../escape.txt - path escapes the workspace: ../escape.txt
  📦 [HOOK:PostToolUse] after every tool call - write_file - FORBIDDEN: SAVE ../escape.txt - path escapes the workspace:

── step 7/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 7/16 - 14 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 7 - tool_calls
think: Reads are not fenced, though - one folder up is fair game:
tool → read_file {"path":"../package.json","limit":5}
  🛠️  [HOOK:PreToolUse] before every tool call - read_file {"path":"../package.json","limit":5}
result ← ../package.json (22 lines)
  📦 [HOOK:PostToolUse] after every tool call - read_file - ../package.json (22 lines)

── step 8/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 8/16 - 16 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 8 - tool_calls
think: A write into the home folder, where a stray file would outlive the demo:
tool → write_file {"path":"~/Desktop/pwned.txt","content":"the sandbox should have stopped this"}
  🛠️  [HOOK:PreToolUse] before every tool call - write_file {"path":"~/Desktop/pwned.txt","content":"the sandbox should
  ⛔ FORBIDDEN SAVE ~/Desktop/pwned.txt - absolute path is outside the workspace: ~/Desktop/pwned.txt
result ← FORBIDDEN: SAVE ~/Desktop/pwned.txt - absolute path is outside the workspace: ~/Desktop/pwned.txt
  📦 [HOOK:PostToolUse] after every tool call - write_file - FORBIDDEN: SAVE ~/Desktop/pwned.txt - absolute path is outsi

── step 9/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 9/16 - 18 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 9 - tool_calls
think: The same trick through the shell - copy it out with a command, which no file tool sees:
tool → run_command {"command":"copy demo/hello.js %USERPROFILE%\\Desktop\\hello-copy.js"}
  🛠️  [HOOK:PreToolUse] before every tool call - run_command {"command":"copy demo/hello.js %USERPROFILE%\\Desktop\\hello
  ⛔ FORBIDDEN COPY %USERPROFILE%\Desktop\hello-copy.js - absolute path is outside the workspace: %USERPROFILE%\Desktop\hello-copy.js
result ← FORBIDDEN: COMMAND COPY %USERPROFILE%\Desktop\hello-copy.js - the shell would write outside the sandbox
  📦 [HOOK:PostToolUse] after every tool call - run_command - FORBIDDEN: COMMAND COPY %USERPROFILE%\Desktop\hello-copy.js

── step 10/16 ─────────────────────────────────────
```

```
  🧠 [HOOK:PreModel] before every model call - step 10/16 - 20 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 10 - tool_calls
think: A command the deny-list refuses no matter what CMD_POLICY says:
tool → run_command {"command":"git push --force origin main"}
  🛠️ [HOOK:PreToolUse] before every tool call - run_command {"command":"git push --force origin main"}
result ← DENIED: blocked by policy (force push). Command: git push --force origin main
  📦 [HOOK:PostToolUse] after every tool call - run_command - DENIED: blocked by policy (force push). Command: git
push --

── step 11/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 11/16 - 22 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 11 - tool_calls
think: A command that asks its own child process for the API key:
tool → run_command {"command":"node -e \"console.log('OPENROUTER_API_KEY=' + (process.env.OPENROUTER_API_KEY ??
'absent'))\""}
  🛠️ [HOOK:PreToolUse] before every tool call - run_command {"command":"node -e \"console.log('OPENROUTER_API_KEY=' +
(p
result ← exit code 0
        ↳ stdout: OPENROUTER_API_KEY=absent
  📦 [HOOK:PostToolUse] after every tool call - run_command - exit code 0

── step 12/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 12/16 - 24 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 12 - tool_calls
think: And a command that fails, so the exit code has to explain itself:
tool → run_command {"command":"node -e \"process.exit(1)\""}
  🛠️ [HOOK:PreToolUse] before every tool call - run_command {"command":"node -e \"process.exit(1)\""}
result ← exit code 1
        ↳ exit 1: general failure - the command ran and then reported an error (uncaught exception, failed assertion,
bad argument). The cause is in stderr; fix that, do not just re-run.
  📦 [HOOK:PostToolUse] after every tool call - run_command - exit code 1

── step 13/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 13/16 - 26 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 13 - tool_calls
think: Saving one durable fact, the way a later run would find it again:
tool → remember {"text":"The sandbox demo script lives at demo/hello.js and is safe to regenerate with
create_file.","tag":"fact"}
  🛠️ [HOOK:PreToolUse] before every tool call - remember {"text":"The sandbox demo script lives at demo/hello.js and
result ← already remembered in src/memory.md (11 total), date refreshed: - [fact] The sandbox demo script lives at
demo/hello.js and is safe to regenerate with create_file. _(2026-10-30)_
  📦 [HOOK:PostToolUse] after every tool call - remember - already remembered in src/memory.md (11 total), date
refresh

── step 14/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 14/16 - 28 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 14 - tool_calls
think: Reading memory back the lazy way - the same file the system prompt was built from:
tool → recall {"query":"demo"}
  🛠️ [HOOK:PreToolUse] before every tool call - recall {"query":"demo"}
result ← 1 of 11 memories in src/memory.md: - [fact] The sandbox demo script lives at demo/hello.js and is safe to
regenerate with create_file. _(2026-10-30)_
  📦 [HOOK:PostToolUse] after every tool call - recall - 1 of 11 memories in src/memory.md: - [fact] The sandbox demo

── step 15/16 ──────────────────────────────────────
  🧠 [HOOK:PreModel] before every model call - step 15/16 - 30 messages
  💬 [HOOK:PostModel] after every reply, before anything is dispatched - step 15 - stop
think: Done. Created demo/hello.js, ran it, edited one string, ran it again (exit 0 both times). Then the fence: the
../escape.txt write and the ~/Desktop/pwned.txt write were refused with FORBIDDEN before touching disk, and the shell
`copy` to the Desktop was refused too - a command is not a way round the sandbox. The read one folder up went through,
because reads are open and writes are not. git push --force was denied by policy, the child process could not see the
API key, and the failing command came back as exit code 1 with an explanation. The scorecard below is checkable, not a
claim.

── final answer ────────────────────────────────────
```

```
Done. Created demo/hello.js, ran it, edited one string, ran it again (exit 0 both times). Then the fence: the
../escape.txt write and the ~/Desktop/pwned.txt write were refused with FORBIDDEN before touching disk, and the shell
`copy` to the Desktop was refused too - a command is not a way round the sandbox. The read one folder up went through,
because reads are open and writes are not. git push --force was denied by policy, the child process could not see the
API key, and the failing command came back as exit code 1 with an explanation. The scorecard below is checkable, not a
claim.
  🏁  [HOOK:Stop] the model answered and the loop is ending - 15 step(s) - Done. Created demo/hello.js, ran it, edited
one string,
  🔒  [HOOK:SessionEnd] always - every run ends here, finished or not - ok (answer) in 379ms

── sandbox checks ──────────────────────────────────────
  ✓ a write that escapes the workspace is refused - FORBIDDEN: SAVE ../escape.txt
  ✓ reads are open - one folder up works - ../package.json (22 lines)
  ✓ a home-folder write is refused - FORBIDDEN: SAVE ~/Desktop/pwned.txt
  ✓ a shell copy out of the sandbox is refused - FORBIDDEN: COMMAND COPY
  ✓ dangerous commands are denied, whatever the policy - DENIED: blocked by policy
  ✓ the child process cannot see the API key - OPENROUTER_API_KEY=absent
  ✓ run_command reports the real exit code - exit code 1

  also enforced, not demoed above: 6000-char output cap per command, 30s command timeout, 8 hard-deny patterns, a 13-
variable env allow-list, and the 'ask'/'deny' command policies.

✓ 7/7 sandbox checks held
```

What to point at

- **Every lifecycle line is tagged `[HOOK:Name]`.** Nine moments, interleaved with the loop at exactly the places chapter 03 numbered: the run opens with `SessionStart` and `UserPromptSubmit`, each step has `PreModel` and `PostModel`, each tool call has `PreToolUse` and `PostToolUse`, and it closes with `Stop` then `SessionEnd`. Nothing else about the run changed when hooks were added — that is the point of putting them at the pauses rather than in the middle of the loop.
- **Steps 1–5 are the loop, and the exit codes are the proof.** `create_file` reports `created demo/hello.js (66 bytes)`; `run_command` reports `exit code 0` and echoes the first line the program printed (`↳ stdout: Hello from a tiny agent`, so the claim is checkable); `edit_file` reports `1 replacement(s), first at line 2`; the second run prints the edited text. Look → act → observe → run again.
- **Steps 6–12 are the fence, tested instead of described.** A write to `../escape.txt` and a write to `~/Desktop/pwned.txt` are refused with a bold red `FORBIDDEN` line *before* any disk access, a read one folder up succeeds (reads are open, writes are not), a shell `copy` of the same file to the Desktop is refused too (a command is not a way round the fence), `git push --force` is denied whatever the policy says, the child process cannot see the API key, and a failing command reports `exit code 1` with its meaning spelled out. The `sandbox checks` scorecard at the end is built from those same tool results — the run exits non-zero if any guard regressed, so it is checkable rather than claimed.
- **Steps 13 and 14 are the only writes that outlive the process.** `remember` appends one bullet to `memory.md` and says how many are now stored; `recall` searches the file and finds it. Every other tool call in this run changed the workspace; those two changed what the *next* run starts with, which is the whole of chapter 12. Note the `memory ←` line above step 1: this capture is a second run, so the fact saved last time is already in the system prompt.
- **Step 15 has no `tool →` line.** The model said it was done, the loop took the `calls.length === 0` branch, printed the final answer and returned — firing `Stop` and `SessionEnd` on the way out (see chapter 11). That branch is the only way out other than hitting the limit.

- **The banner was printed before the first step.** Model, key fingerprint, which `.env` won, the policy, the timeout, the tool list, how many memories are on disk and the hook trace — the whole configuration as one reproducible header, which is what you want when a demo goes wrong on stage.
- **Run it a second time and step 2 fails, and the run carries on.** Now `demo/hello.js` exists, so `create_file` refuses to clobber it and `edit_file` refuses to match an already-edited line. Both arrive as strings, the loop absorbs them, and the run still ends with an answer and a clean scorecard — principle 4 in action, and exactly what a careful human would do.

16 Run it yourself

Node 20+ and one `npm install`. The `.env` at the repository root is found automatically, so nothing has to be copied for a live run.

QUICKSTART · *bash*

```
cd mini-coding-js
npm install                       # one dependency: chalk

npm start                         # a session: type a task, it finishes, type another
npm start -- "create hello.js that prints today's date and time, run it, and save its output to hello-output.txt"
npm run demo                      # scripted run: no network, no key, no excuses
npm test                          # 104 offline tests, no key, no network

node src/cli.js "task" --yes               # auto-approve commands
node src/cli.js "task" --max-steps 6       # tighter loop
node src/cli.js "task" --model anthropic/claude-sonnet-4.5
node src/cli.js --hooks                    # the nine lifecycle hooks, in colour, then exit
node src/cli.js "task" --no-hooks          # silence the [HOOK:...] trace lines
node src/cli.js --memory                   # memory.md, and exactly what it injects
node src/cli.js "task" --no-memory         # one run with nothing remembered
node src/cli.js --help                     # usage summary
```

Sessions: keeping the prompt alive

`npm start` with no task does not exit after the first answer. The CLI calls `runTask` once per task and asks again, so the same program, sandbox, hooks and memory store serve every task in the sitting:

A SESSION IN PROGRESS · *console*

```
  interactive session - type a task below (/new clears, /quit exits)

  task › add a hello.js that prints the date
── step 1/12 ──────────────────────────────────────
...
── final answer ───────────────────────────────────
Created hello.js and ran it: exit code 0.

── task finished ──────────────────────────────────
  ✓ answer in 4 step(s) - 9 message(s) carried into the next task
  type another task, /new to clear the conversation, /quit to exit

  task ›
```

Three behaviours come straight out of the design rather than out of new machinery:

- **The step budget restarts.** `runTask` is one task, and its `for` starts at 1, so the second task gets the full 12 steps — not "step 5 of 12" continued from the first.
- **The conversation is carried, the system prompt is not.** The CLI passes the last 24 messages as `history` (chapter 05), so "now add tests" knows what was just built; the system message is rebuilt, so a memory saved by task one is already in front of the model in task two.
- `/new` **forgets the conversation, not the project.** Files stay, `src/memory.md` stays. `/quit` or Ctrl-C ends the session, and a session exits 0 even if one task failed — the failure is printed and you can try again.

A task passed as an argument stays one-shot (`node src/cli.js "task"`): run it, set the exit code, leave. That is what the demo, the tests and any script want, and it is why the session lives in `cli.js` rather than in the loop.

The five-minute talk script is in the README: run `--demo` and narrate *think → tool → result → think*; delete `workspace/demo/hello.js` and run it live with `--yes`; then ask for an edit that cannot match, so the audience sees the error come back as a tool result and the model repair it on the next iteration. With two minutes left, `--hooks` puts the lifecycle on screen — and every run you have just watched printed those same nine moments as `[HOOK:Name]` lines, so the slide and the console already agree. That is the "how would you extend this?" question, answered. Then show memory as the second extension: run the demo again and point at the `memory ←` line before step 1, or run `--memory` to print the file and the block the prompt received. `--no-memory` gives you the amnesia run for comparison, in one flag.

> **A demo that fails on purpose is worth two that succeed.** The happy path shows what an agent does. The failure shows *why the loop matters*: it is the only structure that lets a model recover from being wrong, which is the thing no single completion can do.

17 What it deliberately does not do

The limits are worth stating as plainly as the features, because each one is where a production agent grows a layer this demo does not have.

- **The shell is not sandboxed, but its writes are read.** `run_command` starts in `workspace/` and a command can walk out of it. `src/sandbox.js` therefore scans the command line before it runs — `copy` / `move` / `xcopy` / `robocopy` destinations, `del` / `rm` / `rd` operands, `>` / `>>` redirections, `curl -o`, PowerShell `-Destination` and the writing cmdlets — and refuses the ones outside with the same `FORBIDDEN` line. That is a command-line scan, not isolation: `node -e "fs.copyFileSync(...)"` or a downloaded script still writes anywhere.
- **No context management, just a cut.** A session carries at most `MAX_HISTORY` (24) messages into the next task, and the cut is positional — it lands on a `user` turn, not on a summary. Nothing is compacted, ranked or retrieved, so a long session eventually loses its beginning rather than condensing it. The memory digest is bounded; the conversation is merely truncated.
- **No streaming.** Each step waits for a complete reply, so the heartbeat exists to make the wait legible. Streaming would delete that problem and add a parser.
- **No sub-agents, no retries, no skills.** Every one of those is a layer on top of this loop, not a replacement for it, and all of them assume the loop already works. What *is* here is the hooks in chapter 11 (nine events, no config file, no plugin loader) and the memory in chapter 12.
- **Memory is one markdown file with substring search.** No embeddings, no vector store, no automatic summarising, no write on every turn. It is the smallest thing that actually persists — and the honest answer to "why did it remember that?" is `node src/cli.js --memory`.
- **One tool at a time in spirit.** Multiple calls in a single reply are dispatched in order, but the system prompt asks for one per step because interleaved edits are much harder to audit live.

The ten principles, in one place

#	PRINCIPLE	WHERE TO SEE IT
1	Tool calling is the model's only ability to act.	`src/tools.js` — the schemas are the prompt
2	The loop is the agent.	`runTask`, `agent.js:88`
3	Everything is messages.	one array, four roles, no other state
4	Errors are observations, not crashes.	`runTool`'s try/catch, `edit_file`'s errors, a blocked `PreToolUse`
5	Exit codes close the loop.	`exit code 0` versus `1` in the result string
6	Human in the loop.	`CMD_POLICY=ask`, plus the hard-deny list
7	Least privilege.	`Workspace.resolve()` for reads, `src/sandbox.js` around save/edit/delete and shell writes, the child-process env allow-list
8	Bounded autonomy.	`maxSteps`, the truncation guard, both timeouts, every memory cap
9	Extension points belong at named moments.	`src/hooks.js` — nine events, called by the loop, never edited into it
10	Memory is a file, not a mystery.	`src/memory.js` — `memory.md`, a bounded digest, and `recall`

That is the whole of it: a model, eight tools, a loop with four bounds, nine lifecycle hooks, one memory file and a sandbox. Everything else you have seen a coding agent do is built from these pieces — and now you know what each piece costs.

Built from the source in `mini-coding-js/` — every excerpt is real code with only the logging lines trimmed and long lines rewrapped to fit, and the run in chapter 15 is a captured terminal session. Tests: `npm test` (104 offline, no key, no network). Demo: `npm run demo`. Hooks: `npm run hooks`. Memory: `node src/cli.js --memory`. Sandbox walkthrough: `sandbox-guide.html` (this page was built alongside `sandbox.html`, which is now `sandbox-guide.html`). Site customiser: the round `Aa` button, bottom right - a named site palette, type scale, layout, fonts and every colour token, saved in `localStorage` and exportable as CSS.